

I. Introduction

Khái niệm và vai trò của Hệ Điều Hành (OS): Hệ điều hành là phần mềm hệ thống giúp thực thi các chương trình ứng dụng và đóng vai trò làm cầu nối giữa phần cứng máy tính và các chương trình ứng dụng

Phân loại Hệ Điều Hành:

- Batch OS:** Xử lý các công việc theo lô mà không có sự tương tác trực tiếp từ người dùng.

- Multiprogramming OS:** Chạy nhiều chương trình cùng lúc để tối ưu hóa việc sử dụng CPU.

- Time-sharing OS (Multitasking OS):** Chia sẻ CPU giữa các chương trình, cho phép nhiều người dùng tương tác nhanh chóng.

- Parallel OS:** Sử dụng nhiều bộ xử lý để tăng tốc độ xử lý.

- Distributed OS:** Liên kết các hệ thống độc lập qua mạng để chia sẻ tài nguyên.

- Real-time OS (RTOS):** Đáp ứng yêu cầu thời gian thực, phổ biến trong các hệ thống điều khiển công nghiệp.

- Embedded OS:** Dùng trong các thiết bị nhỏ gọn như điện thoại, đồng hồ thông minh với tài nguyên giới hạn.

- Các thành phần chính của Hệ Điều Hành:**

- Quản lý tiến trình và tiểu trình:** Điều khiển các quá trình chạy song song và cơ chế giao tiếp giữa chúng.

- Quản lý bộ nhớ:** Cung cấp quản lý không gian bộ nhớ, đảm bảo tính bảo mật và tối ưu hóa sử dụng bộ nhớ.

- Quản lý tệp và đĩa:** Tổ chức, lưu trữ và truy xuất tệp.

- Quản lý thiết bị I/O:** Làm trung gian giữa phần mềm và phần cứng I/O.

- Giao diện hệ thống (System Calls):** Tạo điều kiện cho các chương trình ứng dụng giao tiếp với hệ điều hành thông qua các API.

- Cấu trúc Hệ Điều Hành:**

- Simple Structure:** Cấu trúc đơn giản cho phép ứng dụng truy cập trực tiếp phần cứng, nhưng dễ gây rủi ro. – **MSDOS**

- Monolithic Structure:** Tất cả các thành phần OS nằm trong một module, đơn giản nhưng khó duy trì. – **UNIX**

- Layered Structure:** Các thành phần được tổ chức theo lớp, giúp dễ duy trì và phát triển. – **MULTICS và Window NT**

- Microkernel:** Lõi hệ điều hành chỉ chứa các thành phần cần thiết, các thành phần khác chạy ở chế độ người dùng. – **MacOS và Window**

- Hybrid Structure:** Kết hợp các cấu trúc khác nhau để đạt hiệu suất, bảo mật và tiện dụng. – **Window NT (win10, win 11), MacOS và Linux**

- I. Process and inter-process communication**
Process Concepts (Khái niệm tiến trình):

- Process vs. Program:** Tiến trình là một chương trình đang chạy, trong khi chương trình là mã nguồn hoặc mã thực thi. Tiến trình là một thực thể động và thay đổi trong quá trình thực thi, được tải vào bộ nhớ và sử dụng tài nguyên hệ thống.

- Process States (Trạng thái tiến trình):** Một tiến trình có thể ở các trạng thái như New (mới tạo), Ready (sẵn sàng), Running (đang chạy), Blocked (chờ tài nguyên), và Exit (kết thúc).



- Process Address Space (Không gian địa chỉ của tiến trình):** Gồm mã chương trình, các biến toàn cục và cục bộ, các đối tượng được cấp phát động, và tài nguyên liên kết khi tiến trình chạy.

- Process Features (Đặc tính của tiến trình):**

- Process Control Block (PCB):** Là một cấu trúc dữ liệu chứa thông tin về tiến trình như ID, trạng thái, con trỏ, CPU registers, và bộ đếm chương trình. PCB nằm trong không gian địa chỉ của hệ điều hành và chỉ truy cập được trong chế độ kernel.

- Context Switch (Chuyển đổi ngữ cảnh):** Khi hệ điều hành chuyển CPU từ tiến trình này sang tiến trình khác, hệ thống phải lưu và tải lại trạng thái của tiến trình, dẫn đến chi phí tài nguyên hệ thống.

- Process Operations (Các thao tác với tiến trình):**

- Process Creation (Tạo lập tiến trình):** Tiến trình cha có thể tạo ra các tiến trình con thông qua các lệnh hệ thống như fork (Unix) và CreateProcess (Windows). Các tiến trình con có thể sao chép không gian bộ nhớ của cha hoặc tải một chương trình khác để thực thi.

- Process Termination (Kết thúc tiến trình):** Tiến trình có thể tự kết thúc sau khi hoàn thành nhiệm vụ hoặc bị kết thúc bởi tiến trình cha hoặc hệ điều hành. Khi kết thúc, nó có thể trở thành một tiến trình zombie (đã kết thúc nhưng chưa giải phóng PCB) hoặc bị gắn lại cho tiến trình gốc trong trường hợp cha bị hủy bỏ trước.

- Inter-Process Communication (IPC) - Giao tiếp giữa các tiến trình:**

- Pipes (Đường ống):** Là phương thức truyền thông một chiều giữa hai tiến trình, gồm các loại như ordinary pipes (yêu cầu quan hệ cha-con) và named pipes (không yêu cầu quan hệ cha-con).

- Shared Memory (Vùng nhớ chia sẻ):** Các tiến trình có thể chia sẻ một vùng nhớ chung để trao đổi dữ liệu. Tuy nhiên, cần có cơ chế đồng bộ hóa để đảm bảo tính toàn vẹn dữ liệu khi nhiều tiến trình truy cập.

- Socket:** Sử dụng cho giao tiếp giữa các tiến trình trên các máy khác nhau, dựa trên mô hình client-server. UDP socket cho phép giao tiếp không kết nối, nhanh nhưng ít tin cậy, trong khi TCP socket yêu cầu kết nối nhưng đảm bảo tính toàn vẹn dữ liệu.

II. CPU Scheduling

Basic Concepts (Khái niệm cơ bản):

- CPU-I/O Burst Cycle:** Quá trình thực thi tiến trình bao gồm các chu kỳ CPU burst (thực hiện tính toán) và I/O burst (thực hiện nhập/xuất). Việc định thời CPU liên quan đến việc sắp xếp các tiến trình trong các chu kỳ này để tối ưu hiệu suất.

- Schedulers (Bộ định thời):** Hệ điều hành sử dụng ba loại bộ định thời:
 - Long-term scheduler (định thời dài hạn) quyết định tải tiến trình nào từ job pool vào bộ nhớ.
 - Short-term scheduler (định thời ngắn hạn) chọn tiến trình từ hàng đợi sẵn sàng để thực thi.
 - Medium-term scheduler (định thời trung hạn) quản lý việc swapping, đưa tiến trình vào và ra khỏi bộ nhớ để tối ưu sử dụng CPU.

- Scheduling Criteria (Tiêu chí định thời):**

- Các tiêu chí để đánh giá hiệu quả của một thuật toán định thời bao gồm:**
 - CPU Utilization (Tối đa hóa sử dụng CPU).
 - Throughput (Tối đa hóa số lượng tiến trình hoàn thành trên một đơn vị thời gian).
 - Turnaround Time (Thời gian hoàn thành một tiến trình từ lúc bắt đầu đến khi kết thúc).
 - Waiting Time (Thời gian chờ trong hàng đợi sẵn sàng).

- Response Time (Thời gian từ lúc yêu cầu được gửi đến khi có phản hồi đầu tiên).

Scheduling Types (Các loại định thời):

- Nonpreemptive Scheduling (Định thời không phân chia):** Tiến trình đang chạy giữ CPU cho đến khi nó hoàn thành hoặc chuyển sang trạng thái chờ.

- Preemptive Scheduling (Định thời có phân chia):** Tiến trình đang chạy có thể bị tạm dừng để nhường CPU cho một tiến trình ưu tiên cao hơn.

Scheduling Algorithms (Thuật toán định thời):

- First-Come, First-Served (FCFS):** Thuật toán đơn giản chọn tiến trình đầu tiên trong hàng đợi để thực thi. Tuy nhiên, nó không tính đến độ ưu tiên và có thể dẫn đến thời gian chờ dài dãi với tiến trình có burst time ngắn.

- Round-Robin (RR):** Thuật toán định thời phân chia với time quantum cố định, lần lượt chuyển CPU qua từng tiến trình. Ưu điểm là công bằng và giảm thời gian chờ của các tiến trình có burst time ngắn, nhưng cần chọn thời gian quantum hợp lý.

- Shortest-Job-First (SJF):** Chọn tiến trình có thời gian thực thi ngắn nhất. Phiên bản không phân chia để dẫn đến starvation (tiến trình có burst time dài có thể bị chờ vô thời hạn). Phiên bản 0 độc quyền là Shortest-Remaining-Time-Next (SRTN).

- Priority Scheduling:** Mỗi tiến trình có độ ưu tiên và thuật toán sẽ chọn tiến trình có độ ưu tiên cao nhất. Có hai phiên bản: độc quyền và 0 độc quyền.

- Multilevel Queue Scheduling:** Phân chia hàng đợi sẵn sàng thành nhiều hàng đợi nhỏ, mỗi hàng đợi có một mức độ ưu tiên khác nhau và sử dụng thuật toán định thời riêng.

- Turnaround Time (TT) = (Completion Time – Arrival Time)**
Waiting Time (WT) = (TT – CPU Burst)
WT(RL) = WT(total) – WT(R1) – WT(R2) – ...
Advanced Scheduling Techniques (Các kỹ thuật định thời nâng cao):

- Guaranteed Scheduling:** Đảm bảo mỗi tiến trình sẽ nhận được một lượng CPU nhất định.

- Lottery Scheduling:** Sử dụng vé số để chọn tiến trình, mỗi vé là một lần CPU được chia cho tiến trình tương ứng, đảm bảo tính công bằng.

- Fair-Share Scheduling:** Định thời dựa trên phân bổ tài nguyên công bằng cho các nhóm người dùng hoặc tiến trình.

III. Thread

Thread Concepts (Khái niệm về luồng):

- Definition of Thread:** Luồng (thread) là một đường thực thi trong một tiến trình, được gọi là tiểu trình hay luồng nhẹ. Mỗi tiến trình có thể bao gồm nhiều luồng, cho phép thực thi đồng thời.

- Single-threaded vs. Multi-threaded Processes:** Tiến trình đơn luồng chỉ có một luồng thực thi, trong khi tiến trình đa luồng có nhiều luồng có thể thực thi đồng thời, giúp tăng hiệu suất, đặc biệt trong các hệ thống đa xử lý.

- Advantages of Threads:** Luồng giúp tăng tính responsiveness (tính phản hồi nhanh), chia sẻ tài nguyên hiệu quả, giảm chi phí tạo và quản lý, và tăng khả năng mở rộng (scalability) nhờ khả năng thực thi trên nhiều bộ xử lý hoặc lõi xử lý.

Thread Models (Các mô hình luồng):

- User-level Threads:** Được quản lý hoàn toàn ở không gian người dùng mà không cần sự can thiệp của hệ điều hành, giúp giảm chi phí tạo và chuyển đổi ngữ cảnh. Tuy nhiên, nhược điểm là khi một luồng bị chặn thì cả tiến trình bị chặn.
- Kernel-level Threads:** Được hệ điều hành quản lý ở không gian kernel. Khi một luồng bị chặn, hệ điều

hành có thể chuyển sang luồng khác trong cùng tiến trình, đảm bảo tính liên tục. Tuy nhiên, quản lý luồng trong kernel tương đối chậm và ít hiệu quả hơn do yêu cầu OS can thiệp.

- Hybrid Models:** Kết hợp ưu điểm của cả luồng ở cấp người dùng và cấp kernel để cải thiện hiệu suất và tính linh hoạt. Có nhiều mô hình như:

- Many-to-One Model: Nhiều luồng người dùng ánh xạ tới một luồng kernel.
- One-to-One Model: Mỗi luồng người dùng tương ứng với một luồng kernel.
- Many-to-Many Model: Nhiều luồng người dùng ánh xạ tới nhiều luồng kernel.

Thread Management (Quản lý luồng):

- User-level vs. Kernel-Level Management**

- Thread Scheduling (Định thời luồng):** Luồng có thể được định thời ở cấp người dùng hoặc kernel. Với luồng cấp người dùng, việc định thời chỉ diễn ra trong không gian người dùng mà không cần OS can thiệp, giúp cải thiện hiệu suất.

- Thread Libraries (Thư viện luồng):** Các thư viện luồng phổ biến cung cấp API cho việc tạo và quản lý luồng, như:

IV. Synchronisation

Critical Section Problem (Vấn đề miền gang):

- Race Conditions:** Xảy ra khi nhiều tiến trình hoặc luồng truy cập và thay đổi tài nguyên chia sẻ cùng một lúc. Để tránh các vấn đề tranh chấp, chỉ một tiến trình được phép vào miền gang tại một thời điểm.

- Critical Section (Miền gang):** Là đoạn mã mà nhiều tiến trình hoặc luồng có thể truy cập tài nguyên chia sẻ cùng một lúc. Để tránh các vấn đề tranh chấp, chỉ một tiến trình được phép vào miền gang tại một thời điểm.

- Synchronization Requirements (Yêu cầu về đồng bộ hóa):**

- Mutual Exclusion (Độc quyền truy xuất):** Chỉ một tiến trình hoặc luồng có thể vào miền gang tại một thời điểm.

- Progress (Tiến triển):** Nếu không có tiến trình nào trong miền gang, các tiến trình muốn vào phải có khả năng tiến vào mà không bị cản trở bởi tiến trình khác.

- Bounded Waiting (Giới hạn thời gian chờ):** Tiến trình không nên chờ vô hạn để vào miền gang, đảm bảo công bằng giữa các tiến trình.

- Synchronization Solutions (Các giải pháp đồng bộ hóa):**

- Busy Waiting (Giải pháp bận chờ):**

- Lock Variable: Sử dụng một biến để đánh dấu quyền truy cập miền gang, nhưng không đảm bảo hoàn toàn về mutual exclusion.

- Strict Alternation: Đảm bảo các tiến trình luân phiên vào miền gang, nhưng có thể gây cản trở nếu một tiến trình bên ngoài miền gang.

- Peterson's Solution: Đảm bảo mutual exclusion và tránh tình trạng bận chờ nhưng có thể làm lãng phí CPU.

- Hardware Solutions: Sử dụng giải pháp khóa phần cứng như Test-and-Set Lock (TSL) và Interrupt Disabling để đảm bảo một tiến trình duy nhất vào miền gang.

- Sleep and Wakeup Solutions (Giải pháp ngủ và đánh thức):**

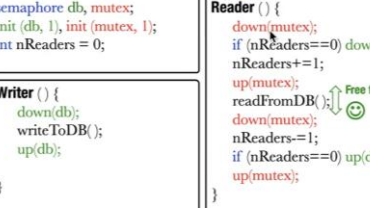
- Semaphores: Sử dụng các biến nguyên có hai thao tác cơ bản là down (P) và up (V) để điều khiển truy cập miền gang. Có hai loại semaphores là binary semaphore (mutex) và counting semaphore.

- Monitor: Một cấu trúc lập trình điều khiển truy cập vào dữ liệu chia sẻ. Monitor bao gồm các biến điều

kiện và các thủ tục để đảm bảo chỉ một tiến trình được thực thi trong monitor tại một thời điểm.

On tap:

lồng bộ hoá (Synchronization): Readers-Writers P1



câu 2

Cho đoạn code bên dưới được chia sẻ bởi hai tiến trình P1, P2. Giải pháp nào để đảm bảo độc quyền truy xuất không. Giải thích.

```
while (TRUE) {
    int i = 1;
    flag[i] = TRUE; turn = i;
    while (turn == i && flag[i] == TRUE);
    critical_section(i);
    flag[i] = FALSE;
    noncritical_section(i);
}
```

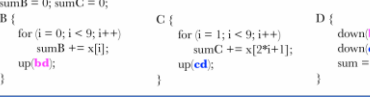
Xét tình huống giả sử: P1, P2 lần lượt có i = 0, 1:
• P1 thực thi gain flag[0] = 1
• Hệ thống lấy lại CPU cho P2
• P2 thực thi gain flag[1] = 1, turn = 1
• P2 vào miền gang
• Hệ thống lấy lại CPU cho P1
• P1 gain turn = 0
• P1 vào miền gang
→ Cả P1 và P2 đều ở trong miền gang

Cho mảng sara: int s[20];

Dùng semaphore đồng bộ hoá cho 3 threads B, C, D thực hiện các nhiệm vụ sau trên tiêu chí khai thác tối đa khả năng xử lý song song, chia sẻ tài nguyên đồng chung giữa các threads.

- a. B tính tổng giá trị của phần tử của mảng x có chỉ số chẵn
- b. C tính tổng giá trị của phần tử của mảng x có chỉ số lẻ
- c. D tính tổng giá trị của phần tử của mảng x dựa trên kết quả từ B và C

Giải sara: B, C, D cùng đến hệ thống tại 1 thời điểm, có thể kết thúc công việc mà không cần chờ đợi nhau.



V. File management

Giao diện hệ thống tệp (File System Interface)

- Khái niệm tệp tin:** Tệp là đơn vị lưu trữ logic chứa dữ liệu liên quan trên ổ đĩa, bao gồm các loại tệp thông thường, tệp thư mục, tệp đặc biệt (đại diện cho thiết bị vật lý).

- Phương thức truy cập:** Gồm truy cập tuần tự, truy cập ngẫu nhiên và truy cập chuỗi.

- Cấu trúc tệp:** Bao gồm các phần tử byte hoặc chuỗi bản ghi cố định, hoặc cấu trúc cây.

- Thư mục và quản lý thư mục:** Thư mục chứa các tệp hoặc thư mục con, có thể truy cập bằng đường dẫn tuyệt đối hoặc tương đối.

- Tổ chức thư mục:** Các kiểu tổ chức như thư mục đơn cấp, hai cấp, cây thư mục, và thư mục dạng đồ thị nhằm tối ưu hóa tìm kiếm và chia sẻ tệp tin.

- Bảo vệ và phân quyền truy cập:** Sử dụng quyền đọc, ghi, thực thi cho ba nhóm người dùng (chủ sở hữu, nhóm, toàn bộ).

2. Cài đặt hệ thống tệp (File System Implementation)

- Góc nhìn của người dùng và hệ thống:** Tệp tin được truy cập dễ dàng mà không cần biết cấu trúc lưu trữ vật lý; hệ thống xử lý việc ánh xạ tệp lên ổ đĩa.

- Hệ thống tệp phổ biến:** FAT, NTFS (Windows); ext2, ext3, ext4 (Linux); HFS, APFS (macOS); FAT32 và exFAT cho thiết bị di động.

- Phương pháp phân bổ:**
 - Contiguous Allocation: Tệp lưu trong các khối liên tiếp.
 - Linked List Allocation: Các khối không liên kề, liên kết qua con trỏ.

- **Indexed Allocation:** Khối chỉ số chứa con trỏ tới khối dữ liệu.

- **Quản lý không gian trống:** Sử dụng bit map, danh sách liên kết hoặc đếm số khối trống liên tiếp để quản lý không gian đĩa.

3. Cấu trúc hệ thống tệp (File System Structure)

- **Bố trí đĩa (Disk Layout):** Cấu trúc phân vùng đĩa và bảng Master Boot Record (MBR) để khởi động hệ điều hành.

- MBR chứa thông tin quan trọng để khởi động hệ thống và quản lý các phân vùng trên ổ đĩa.

- **Các cấu trúc đặc trưng:**

- **FAT12: 32MB**
- **FAT16: 4GB**
- **FAT32: 8TB**

- **FAT Structure:** Dùng bảng FAT cho phương pháp liên kết khối.
- **NTFS Structure:** Hỗ trợ bảo mật, mã hóa và quản lý khối lượng lớn.
- **UNIX I-node Structure:** Dùng các khối trực tiếp và gián tiếp để ánh xạ tệp.

4. Quản lý ổ đĩa (Disk Management)

- **Cấu trúc ổ đĩa và định dạng:** Cấu trúc đĩa bao gồm trục, đầu đọc/ghi, và cấu hình phân vùng.

- **Định dạng và phân vùng ổ đĩa:** Định dạng mức thấp và cao, cùng với chia phân vùng để tạo hệ thống tệp.

- **Phương pháp truy cập ổ đĩa:** Sử dụng địa chỉ dạng CHS (Cylinder-Head-Sector) hoặc LBA (Logical Block Addressing) để xác định vị trí dữ liệu.

5. Thuật toán lập lịch đĩa (Disk Scheduling Algorithms)

- **Các thuật toán:** Bao gồm FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK để tối ưu hóa thời gian truy cập đĩa, phù hợp với nhu cầu và cấu trúc dữ liệu của hệ thống.

Thời gian truy cập đĩa = Thời gian di chuyển đầu đọc (Seek Time) + Thời gian quay (Rotational Time) + Thời gian truyền dữ liệu (Data Transfer Time).

Question#1

A 255-Gb disk has 65,536 cylinders with 255 sectors per track and 512 bytes per sector. How many platters and heads does this disk have? Assuming an average cylinder seek time of 11 msec, average rotational delay of 7 msec and reading rate of 100 MB/sec, calculate the average time it will take to read 400 KB from one sector

255×2^{20} (bytes) = 2^{29} (bytes) * 2^{25} (sectors) * 2^{16} (cylinders/tracks) * N_{heads}
→ $N_{heads} = 2^{25} = 32$
→ $N_{platters} = 32/2 = 16$
Seek time = 11msec
Rotational Time = 7msec
Transfer Time = $(400KB/100MB) \times 1000 = 3.9msec$
→ Disk Access Time = $11 + 7 + 3.9 = 21.9 msec$

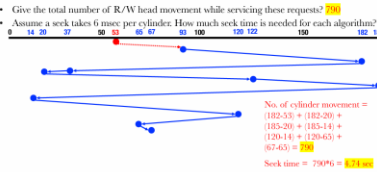
Suppose that a 10-MB file is stored on a disk on the same track (track 50) in consecutive sectors. The disk arm is currently situated over track number 100. How long will it take to retrieve this file from the disk? Assume that it takes about 1 msec to move the arm from one cylinder to the next and about 5 msec for the sector where the beginning of the file is stored to rotate under the head. Also, assume that reading occurs at a rate of 200 MB/s.

Seek time = $(100-50) \times 1 = 50msec$
Rotational Time = 5msec
Transfer Time = $(10/200) \times 1000 = 50msec$
→ Disk access time = $50 + 5 + 50 = 105$

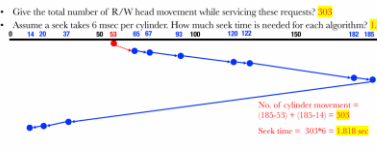
Consider the Unix I-node which uses 12 direct data block addresses, 1 single indirect, 1 double indirect, and 1 triple indirect. The disk block address requires 32 bits and disk block size is 1 KB. What is the maximum size?

No. of pointers per disk block = $1KB/(32bits) = 2^{10}$ (bytes) / 2^{12} (bytes) = $2^8 = 256$ (pointers)
Data block size of 12 direct blocks = $12 \times 1KB = 12KB$
Data block size of 1 single indirect block = $256 \times 1KB = 256KB$
Data block size of 1 double indirect block = $256 \times 256 \times 1KB = 65\,536KB$
Data block size of 1 triple indirect block = $256 \times 256 \times 256 \times 1KB = 16\,777\,216KB$
→ Maximum file size = $(12 + 256 + 65536 + 16777216) = 16\,813\,020KB \sim 16GB$
Nb of data blocks = $12 + 256 + 256^2 + 256^3$ (data blocks)

Consider a disk request for disk blocks on cylinders as follows: 93, 182, 37, 20, 122, 185, 14, 120, 6, 67. The cylinders are numbered from 0 to 199. Assume the current position of R/W head is at 53. For each of FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK disk scheduling algorithms:



Consider a disk request for disk blocks on cylinders as follows: 93, 182, 37, 20, 122, 185, 14, 120, 65. The cylinders are numbered from 0 to 199. Assume the current position of R/W head is at 53. For each of FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK disk scheduling algorithms:



- MBR: thường nằm ở sector logic 0, kích thước 512 bytes.

- Phân vùng (partition): **primary, extended**. Có tối đa 4 phân vùng.

- Boot block + super block (**boot sector**): chứa thông số quan trọng của phân vùng; chứa đoạn chương trình nhỏ để nạp vào HDH khi khởi động máy

Boot sector FAT32

Offset	Số byte	Nội dung
0	3	Jump Code: lệnh nhảy qua vùng ổ cứng số (như FAT)
3	8	OEM ID: tên của nhà sản xuất - version, thường là "MSWIN4.1"
8	2	Số byte trên Sector, thường là 512 (như FAT)
9	1	Số sector trên cluster (như FAT)
10	2	Số sector thuộc vùng Bootsector (như FAT)
11	1	Nội dung bảng FAT, thường là 2 (như FAT)
12	2	Khống định, thường là 0 (số entry của RDET - với FAT)
13	2	Khống định, thường là 0 (số sector của vol - với FAT)
15	1	Loại thuật toán (FIFO hoặc LIFO định cứng - như FAT)
16	2	Khống định, thường là 0 (số sector của bảng FAT - với FAT)
18	2	Số sector của track (như FAT)
1A	2	Số hàng đầu đọc (như FAT)
1C	4	Khống định hoặc mã tài liệu đầu vào (như FAT)
20	4	Số Kích thước volume (như FAT)
24	4	Số Kích thước mỗi bảng FAT
28	2	bit 0 bật: chỉ ghi vào bảng FAT active (có chỉ số là 4 bit đầu)
2A	2	Version của FAT32 trên vol này
2C	4	Cluster bắt đầu của RDET
30	2	Sector chứa thông tin phụ (về cluster trống), thường là 1
32	2	Sector chứa địa chỉ của Boot Sector
34	C	Địa chỉ của các phần mềm sau
40	1	Kích thước vật lý của đĩa trên vol (0 - mềm, 30h: cứng)
41	1	Địa chỉ của
42	1	Kích thước phân vùng HEB
43	4	SerialNumber của Volume
47	B	Volume Label
52	8	Loại FAT, là chuỗi "FAT32"
5A	1A4	Đoạn chương trình khởi tạo ổ nạp HDH khi khởi động máy
1FE	2	Địa chỉ của bảng BootSector Master Boot (bên là AA55h)

- Lưu ý, nếu có nhiều hơn 1 byte cho 1 offset thì thứ tự đọc sẽ đảo ngược từ phải qua trái, trong bảng FAT thì volume = partition.

- Kích thước vùng FAT = Kích thước bảng FAT x số bảng FAT.

- Sector bắt đầu bảng FAT = tổng số sector từ đầu đĩa cứng (khoảng cách từ nơi mô tả vol đến đầu vol) + sector dành cho boot sector.

- Sector bắt đầu RDET = sector bắt đầu bảng FAT + số bảng FAT * Kích thước bảng FAT + (Cluster bắt đầu RDET - 2)*số sector trong 1 cluster.

- **Bảng thư mục gốc: RDET**

- Byte đầu tiên của mỗi entry cho biết:

- 0 – entry trống
- E5h – tập tin chiếm entry này đã bị xóa
- Giá trị khác – thông tin của tập tin thư mục

- Entry chính chứa các thông tin của tập tin Entry chính

Offset (hex)	Số byte	Ý nghĩa
0	8	Tên chính của tập tin - tên bằng mã ASCII
4	3	Tên ngược - mã ASCII
8	1	Thuộc tính trạng thái (R.A.D.V.S.H.R.)
C	1	Địa chỉ ngược
D	3	Giới hạn (mảng): 5; tháng: 4; năm: 1980; 7)
10	2	Ngày tạo (ngày: 5; tháng: 4; năm: 1980; 7)
12	2	Ngày truy cập gần nhất (từ nay trở về)
14	2	Cluster của file - phần trước (20 byte)
16	2	Giới hạn của tập tin (5; tháng: 4; năm: 1980; 7)
18	2	Ngày cập nhật gần nhất (từ nay trở về)
1A	2	Cluster của file - phần sau (20 byte)
1C	4	Kích thước của phần nội dung tập tin

- Entry phụ chỉ chứa tên của tập tin.

Offset	Số byte	Ý nghĩa
0	1	Thứ tự của entry (bắt đầu từ 1)
1	A (10d)	5 ký tự UniCode – bảng mã UTF16
B (11d)	1	Dấu hiệu nhận biết (luôn là 0Fh)
E (14d)	C (12d)	6 ký tự kết tiếp
1C (28d)	4	2 ký tự kết tiếp

- Mỗi phần tử trong RDET là 2 dòng (32 bytes)

- Giá trị của offset B 0F là entry phụ, 10 là thư mục, 20 là tập tin

Bit	Mask	Attribute
0	0x01	Read-only
1	0x02	Hidden
2	0x04	System
3	0x08	Volume label
4	0x10	Subdirectory
5	0x20	Archive
6	0x40	Unused
7	0x80	Unused

Bảng phân vùng: Master Boot Record bao gồm 446 byte code, theo sau là bảng phân vùng với 64 byte và hai byte còn lại được dành riêng cho chữ ký đĩa.

Bytes đầu tiên của phân vùng thể hiện có phải phân vùng boot không

Byte thứ 5 là xác định loại phân vùng 4 byte cuối xác định độ lớn

Loại mã	Ý nghĩa
0x01	FAT12
0x04, 0x06	FAT16
0x07	NTFS hoặc exFAT
0x0B, 0x0C	FAT32
0x82	Linux Swap
0x83	Linux Native (ext2, e
0xEE	GPT Protective MBR

Chuyển đổi sector vật lý và logic:

- Sector vật lý → Sector logic
 $L = (t * st * hd) + (h * st) + (s - 1)$
- Sector logic → Sector vật lý
 $s = (L \text{ mod } st) + 1$
 $t = L \text{ div } (st * hd)$
 $h = (L \text{ div } st) \text{ mod } hd$

Trong đó:

- L : chỉ số sector logic
- st : chỉ số sector/track
- h : chỉ số head
- t : chỉ số track
- s : chỉ số sector vật lý
- hd : chỉ số head/head
- st : chỉ số track/head
- hd : chỉ số head/head

Hd=2, st= 18

Các thuật toán Avoidance sẽ ko để hệ thống rơi vào trạng thái unsafe
+ Safe state: ht dc coi là safe khi tồn tại một chuỗi cấp phát an toàn. Đảm bảo tt đều sao hữu dc tn để hoàn tất.

Deadlock Avoidance đảm bảo hệ thống an toàn sau mỗi lần cấp phát tài nguyên.

Banker's algorithms: available(số lg tài nguyên rảnh trong hệ thống), Max(số lượng tia nguyên tối đa mỗi tiến trình yêu cầu), Allocation(số tài nguyên mỗi tiến trình đang giữ), Need(Số tài nguyên mà tiến trình cần thêm)
Need = Max – Allocation.

Deadlock Avoidance đảm bảo hệ thống an toàn sau mỗi lần cấp phát tài nguyên.

Deadlock Detected and recovery: để DEADLOCK xảy ra sau đó giải quyết bằng cách recovery lại.

Detected: sử dụng wait-for graph và matrix-based detection algorithms. Recovery: loại bỏ process, rollback lại hệ thống tại checkpoint an toàn trc đó.

Thu hồi tài nguyên.

Chương 8: Memory

Virtual address: địa chỉ ảo của từng dòng lệnh trong tiến trình. Virtual address space: không gian địa chỉ của các chương trình. Làm việc bởi CPU và các tiến trình.

Physical address: địa chỉ thực sự được lưu ở bộ nhớ, được lưu dưới RAM . Physical address space: Không gian địa chỉ bộ nhớ.

Address binding(tải định vị): biết được địa chỉ thực sự được lưu như thế nào. Mapping từ address space này sang address space khác. Nhưng cái mà address binding làm: compiler time, load time(link thư viện, load vào bộ nhớ), Execution time (binding chỉ dc map khi nào thực thi, hầu hết hdn hiện tại đều làm ở đây, cũ làm ở 2 trên).

Memory management unit được cài trong CPU, CPU gửi logic address đến MMU và MMU gửi đến physical.

MMU sẽ làm allocation/ deallocation, address binding, protection, sharing.

Contiguous Memory allocation: cấp phát bộ nhớ liên tục. Tiến trình phải dc nạp vào một vùng nhớ liên tục trong vùng nhớ chính. Physical memory dc chia thành các partition(fixed-partition/ dc chia sau khi load tiến trình vào (variable partition/ fixed-partition: memory dc chia thành các partition, các tiến trình vô hàng đợi và sẽ dc đưa đến partition hợp lý).

Lỗi kích thước phân vùng sẽ không phù hợp với tiến trình dc nạp.
variable partition: Các tiến trình sẽ đi vào chỗ còn trống, sau đó đi ra và để lại ô trống đó (hole). Partition dc cấp

phát đúng với kích thước cần (tiến trình cần. Memory allocation strategies(chiến lược cấp phát bộ nhớ): first fit(chọn cái đầu tiên), best fit(chọn cái có kích thước nhỏ nhất có thể chứa tiến trình), worst-fit(chọn cái lớn nhất).

Address protection: Reallocation register(lưu địa chỉ physical nhỏ nhất của tiến trình, Limit register(lưu địa chỉ physis lớn nhất của tiến trình)).
MMU nhận địa chỉ logic sau đó kiểm tra xem có nhỏ hơn limit không, nếu nhỏ hơn thì + reallocation ra physis.

Swapping là việc thay đổi các tiến trình vào ra giữa memory và backing store(fast disk). Tiến trình đang dc thực thi, sắp dc thực thi sẽ đưa vào main memory, khi ko cần nữa thì đẩy ra backing store. Thời gian chuyển đổi = 2* (dung lượng process/ thời gian nạp dung lượng mỗi s).
External Fragmentation(phân mảnh ngoại vi): Bị khi variable partition. Các ô nhớ trống đủ kích thước của process nhưng ko liên tục. Cách giải quyết: Compaction, gom các ô còn trống lại.
External Fragmentation(phân mảnh nội vi): Bị khi fixed-partition. Phần partition bị dư ra khi đưa proc vào ko dc sử dụng. Cách giải quyết: best fit allocation.

Chương 9: Virtual Memory
Dynamic Linking: Việc linking sẽ dc delay đến executed time, khi cần library nào thì load đó.
Dynamic Loading: Việc loading sẽ dc delay đến executed time, cần đoạn mã nào thì load đoạn đó.
Overlay: một vài phần của tiến trình đang thực thi sẽ dc nạp vào bộ nhớ.

Virtual memory cho phép đẩy 1 phần ra backing store. Virtual memory lưu bằng page, nào đang udng thì ở memory khoogn thì ở backing store.
Non-contiguous memory allocation: Các tiến trình được chia thành các phần khác nhau, Các phần này không cần liên tục nhau nhưng một đoạn hoàn chỉnh thì cần liên tục. Bộ nhớ thành: segmentation và paging.

Segmentation: Tiến trình chia thành các segment lưu vào các đoạn bộ nhớ không liên tục. 2 register: base, limit
Address binding: địa chỉ ảo gồm 2 phần <s: segment, d: offset>, offset: 0 - limit-1
Tự nhớ ảo sang nhớ vật lý sẽ có segment table, mỗi dòng sẽ chứa base và limit. Kiểm tra địa chỉ hợp lệ: kiểm tra offset có < limit không, nếu nhỏ hơn lấy offset+base.

Paging: process chia thành các trang, bộ nhớ chia thành các frame. Load trang vào frame, các trang không load liên tục nhưng một trang phải nằm hoàn toàn trong frame. Kích thước Page = kích thước Frame.

Địa chỉ logic <p: page, d: offset>, bộ nhớ chính: <f: frame, d: offset>
Có thể biểu diễn đặc hi logic: <p,d>/ địa chỉ tuyệt đối: p* số lượng d một page + d.

Số bit thành ghi: 16 bit logic: 4 bit offset, 12 bit page.

12 bit vậy lý: 4 bit offset, 8 bit frame.

Page table gồm: Frame number, Valid/invalid,

Protection(readonly/ WR), Caching, Referenced(Cho

trợ dc dc truy cập ở chu kỳ trc ko), Modified(Trang

đó có vừa được thay đổi ko/ Dirtybit.

Từ logic sang vật lý sử dụng bảng trang ra frame giữ

lại offset, nếu invalid gây ra lỗi trang. Ở ra backing

store lưu nạp vào memory reset bảng trang và truy

vấn lại.

Thời gian truy cập bộ nhớ hiệu dụng: EAT = (1-p) * tm

+ p * tp. tp: tg lỗi, tm thời gian truy cập memory.

Translation Lookaside Buffer (TLBs): Lưu bảng phân

trang ở cache giúp tăng tốc độ truy vấn. Nhưng trang

đã có trong TLB gọi là TLB hit, còn ko có thì TLB miss.

EAT = h*(tc+tm) + (1-h) * (tc+2trm)

Paging with large address space: Sử dụng bảng trang

đa cấp

Two level paging in 32-bit system: p2 p1 d p2: 10bit,

p1: 10bit, d: 12bit.

Còn có thể dung hash table, inverted page table,